ADA LOGICS

Scorecard Security Audit

In collaboration with OSTIF and the Scorecard maintainers

Adam Korczynski, David Korczynski, Ada Logics

8th October 2025

About Ada Logics

Ada Logics is a software security company founded in Oxford, UK, 2018 and is now based in London. We are a team of dedicated, pragmatic security engineers and security researchers that work hands-on with code auditing, security automation and security tooling.

We are committed open source contributors and we routinely contribute to state of the art security tooling in the fuzzing domain such as advanced fuzzing tools like Fuzz Introspector and continuous fuzzing with OSS-Fuzz. For example, we have contributed to fuzzing of hundreds of open source projects by way of OSS-Fuzz. We regularly perform security audits of open source software and make our reports publicly available with findings and fixes, and we have audited many of the most widely used cloud native applications.

Ada Logics contributes to solving the challenge of securing the software supply-chain. To this end, we develop the tooling and infrastructure needed for ensuring a secure software development lifecycle, and we deploy these tools to critical software packages. On the tooling and infrastructure side, we contribute to projects such as the OpenSSF Scorecard project as well as the Sigstore projects like SLSA and Cosign.

Ada Logics helps some of the most exposed organisations secure their software, analyse their code and increase security automation and assurance, and if you would like to consider working with us please reach out to us via our website.

We write about our work on our blog. You can also follow Ada Logics on Linkedin, Twitter and Youtube.

Ada Logics ltd
71-75 Shelton Street,
WC2H 9JQ London,
United Kingdom

About Open Source Technology Improvement Fund

The Open Source Technology Improvement Fund (OSTIF) is dedicated to resourcing and managing security engagements for open source software projects through partnerships with corporate, government, and non-profit donors. We bridge the gap between resources and security outcomes, while supporting and championing the open source community whose efforts underpin our digital landscape.

Over the past ten years, OSTIF has been responsible for the discovery of over 800 vulnerabilities, (121 of those being Critical/High), over 13,000 hours of security work, and millions of dollars raised for open source security. Maximizing output and security outcomes while minimizing labor and cost for projects and funders has resulted in partnerships with multi-billion dollar companies, top open source foundations, government organizations, and respected individuals in the space. Most importantly, we've helped over 120 projects and counting improve their security posture.

Our directive is to support and enrich the open source community through providing public-facing security audits, educational resources, meetups, tooling, and advice. OSTIF's experience positions us to be able to share knowledge of auditing with maintainers, developers, foundations, and the community to further secure our infrastructure in a sustainable manner.

We are a small team working out of Chicago, Illinois. Our website is ostif.org. You can follow us on social media to keep up to date on audits, conferences, meetups, and opportunities with OSTIF, or feel free to reach out directly at contactus@ostif.org or our Github.

Derek Zimmer, Executive Director
Amir Montazery, Managing Director
Helen Woeste, Communications and Community Manager
Tom Welter, Project Manager

Contents

Audit contacts

Contact	Role	Organisation	Email
Adam Korczynski	Auditor	Ada Logics Ltd	adam@adalogics.com
David Korczynski	Auditor	Ada Logics Ltd	david@adalogics.com
Spencer Schrock	Scorecard Maintainer	Google	sschrock@google.com
Raghav Kaul	Scorecard Maintainer	Google	raghavkaul@google.com
Stephen Augustus	Scorecard Maintainer	Bloomberg	saugustus2@bloomberg.net
Jeff Mendoza	Scorecard Maintainer	Kusari	jlm@jlm.name
Amir Montazery	Facilitator	OSTIF	amir@ostif.org
Derek Zimmer	Facilitator	OSTIF	derek@ostif.org
Helen Woeste	Facilitator	OSTIF	helen@ostif.org

Introduction

Risk scoring

During the audit we used a simplified risk scoring system that considers risk exposure and risk impact. Exposure is the level at which an issue is exposed to an attacker. Impact is the level of privilege escalation an attacker can obtain by exploiting the security issue. We score both on a scale of 1-5 and add the two scores together for a final combined score. This score determines the severity of security issues. We assign this severity to the issues we find.

Risk Exposure

- 5: The security issue exists in core component(s) and is exposed in all use cases to untrusted input.
- 4: The security issue exists in widely used component(s) and is enabled by default. Users of the component(s) expose the issue by default to untrusted input.
- 3: The issue is exposed to authenticated and/or authorized users only.
- 2: The issue exists in component(s) that users need to enable to be affected.
- 1: The issue is only exposed to trusted users.

Risk Impact

- 5: An attack will have the highest possible impact.
- 4: An attack will have high impact with some constraints or limitations.
- 3: An attack can cause partial harm.
- 2: An attack can result in privilege escalation that will cause limited harm.
- 1: An attack can result in limited privilege escalation but requires further privilege escalation to cause harm.

We score each issue on both scales and then add the scores for a combined total score. The total score is the basis for the overall severity of found issues.

- 10: Critical
- 9 - 8: High
- 7 - 6: Moderate
- 5 - 4: Low
- 3 - 1: Informational

Scope

The audit included the code in the following code repositories:

1. Scorecard-Webapp: https://github.com/ossf/scorecard-webapp
2. Scorecard-Action: https://github.com/ossf/scorecard-action
3. Scorecard-Monitor: https://github.com/ossf/scorecard-monitor
4. Scorecard: https://github.com/ossf/scorecard
5. Allstar: https://github.com/ossf/allstar

The audit was not fixed to a particular commit; we worked constantly against the latest master branches.

Scorecard threat model

This chapter defines the formal threat models for each of the five projects within the scope of this audit: Scorecard-webapp, Scorecard-action, Scorecard-monitor, Scorecard, and Allstar. These threat models are constructed to provide a structured and consistent security perspective for each project, serving as a foundation for targeted manual auditing efforts. By identifying relevant assets, entry points, trust boundaries, and potential adversarial actions, the models support a focused assessment of each system's security posture. This approach ensures that our manual review is guided by a clear understanding of the threats most pertinent to each project's design and operational context.

Scorecard web app

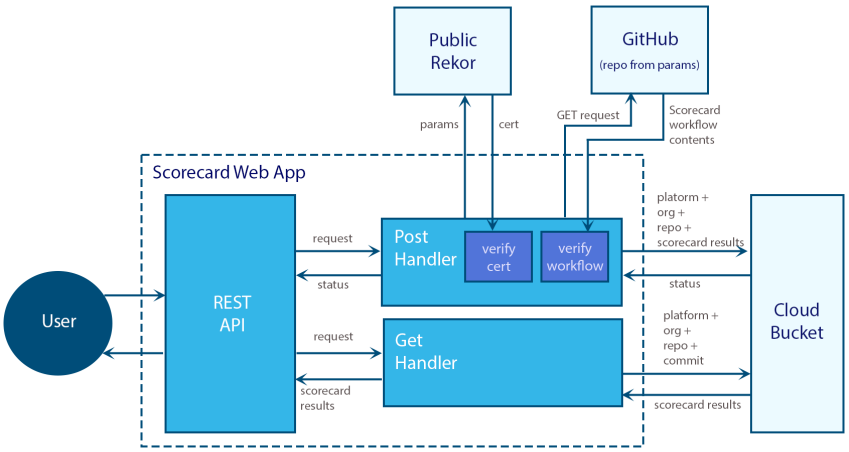
URL: https://github.com/ossf/scorecard-webapp

This is the source code for the scorecard.dev website and API.

In this audit we focus only on the API which has two pieces of functionality/endpoints:

1. GET handler: Retrieves scorecard results for a project.
2. POST handler: Adding/updating scorecard results for a project.

The dataflow looks as such:



The GET handler is relatively simple. Users send a get request to the server, which exposes the GET handler behind a REST API. The REST API forwards the request to the get handler, which retrieves the scorecard results from a cloud bucket and returns the results to the user. The user's initial request contains parameters that identify the project name to fetch Scorecard results for.

The POST handler is more complex than the GET handler. It is also exposed by way of a REST API. Users send a POST request to the REST API with parameters that identify the project as well as scorecard results. The POST handler fetches a certificate from the public Rekor instance and verifies it. Once verified, it verifies the certificate against the parameters of the POST request. Once that is verified, the POST handler sends a request to the project's workflow and verifies it locally. Once that is verified, the POST handler uploads the results to the cloud bucket and returns a successful status code to the user.

Interface enumeration

Scorecard web app has the following interfaces:

#	Interface	Inbound data	Outbound data
1	User	GET/POST request	Scorecard results
2	Public Rekor	Certificate	Parameters
3	Github Repository	Scorecard workflow results	GET request
4	Cloud bucket	HTTP request status/Scorecard results	POST/GET request

Attack impact

Examples of ways malicious threat actors can compromise Scorecard Webapps security.

#	Description	Impact
1	Deny any component from serving other users	Prevent other users from using Scorecard Webapp.
2	Command execution	Leak sensitive data, deliver malicious or wrong results to users, prevent other users from using Scorecard Webapp.
3	Compromise scorecard results	Deliver malicious or wrong results to users.

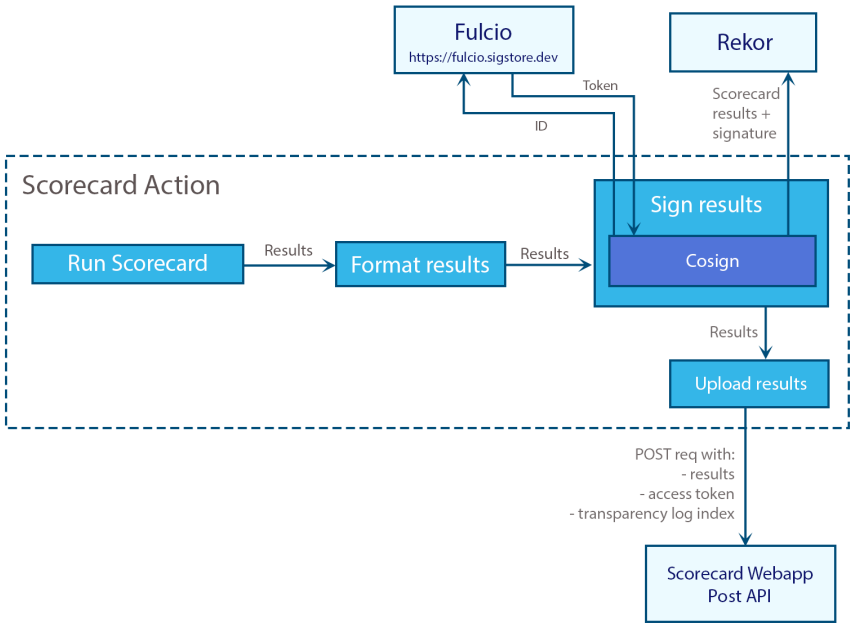
Scorecard Action

URL: https://github.com/ossf/scorecard-action

This is the official GitHub action for running Scorecard in the GitHub CI. It allows projects to run Scorecard and upload the results to Rekor and post the results to the Scorecard Webapp POST handler. As such, a successful run of this action results in uploading verifiable scorecard results for the project.

Internally, when a project invokes the Scorecard action, the action does the following:

1. First, it runs Scorecard on the project [ref]
2. Next, it formats the Scorecard results [ref]
3. Next, it signs the Scorecard results [ref] and uploads the Scorecard results to the public Rekor service [ref]
4. Finally, it sends a POST request with the Scorecard results to the public Scorecard Webapp instance [ref and ref]



Interface enumeration

Scorecard Action has the following interfaces:

#	Interface	Inbound data	Outbound data
1	Public Rekor	none	Scorecard results + cosign signature
2	Scorecard webapp	none	POST request with scorecard results, GH access token

Sensitive data

Scorecard Action processes the following sensitive data:

#	Name	Level of sensitivity	Documentation
1	ID token	High	https://docs.sigstore.dev/certificate_authority/oidc-in-fulcio/
2	GitHub Token	Moderate	https://github.com/ossf/scorecard?tab=readme-ov-file#authentication
3	Scorecard results	Moderate	

The table only includes data that the application processes. It does not include data about its environment, the user, files on the system, etc., which can be sensitive.

Attack impact

Examples of ways malicious threat actors can compromise Scorecard Actions security.

#	Description	Impact
1	Deny Scorecard Action of service	Prevents Scorecard Action from uploading scorecard analysis results which in turn could impact business logic or allow an attacker to hide issues in packages to compromise them.
2	Command execution	Same as above.
3	Hijack scorecard analysis	Upload wrong results.

Scorecard Monitor

URL: https://github.com/ossf/scorecard-monitor

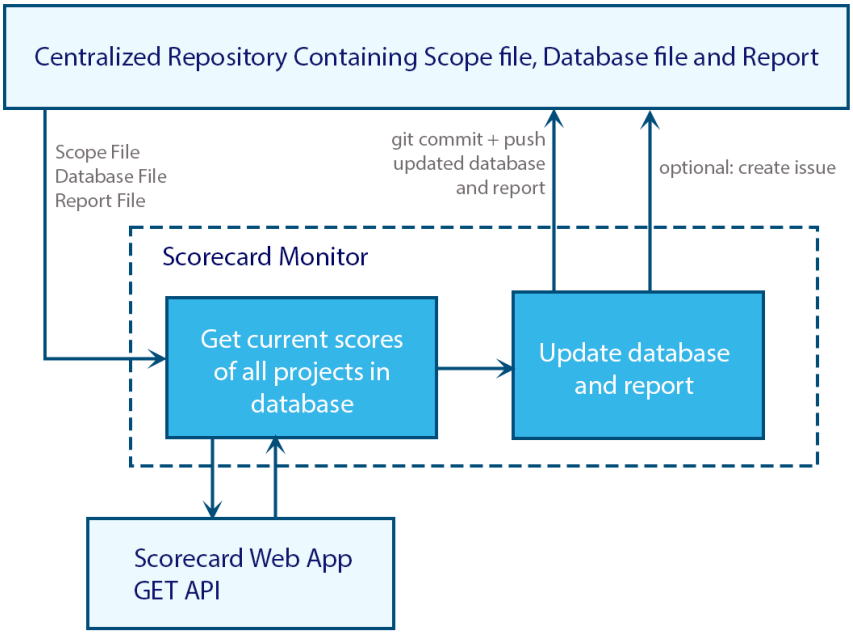
Scorecard Monitor is a CI workflow for GitHub that allows users to monitor Scorecard results for many projects across their organization in a centralized manner. It takes as input a list of repository references and reports that contain Scorecard results, and it outputs a report and can optionally create issues in a centralized GitHub repository about negative changes to any of the projects' Scorecard results.

To use Scorecard Monitor, the use invokes the action as such:

```
24    - name: OpenSSF Scorecard Monitor
25      uses: ossf/scorecard-monitor@a3a9c4cfa0684480ec5f86fa178fc22c4394b69e # v2.0.0-
            beta8
26      with:
27        scope: tools/ossf_scorecard/scope.json
28        database: tools/ossf_scorecard/database.json
29        report: tools/ossf_scorecard/report.md
30        auto-commit: false
31        auto-push: false
32        generate-issue: true
33        report-tags-enabled: true
34        issue-title: "OpenSSF Scorecard Report Updated!"
35        github-token: ${{ secrets.GITHUB_TOKEN }}
36        max-request-in-parallel: 10
37        discovery-enabled: true
38        discovery-orgs: 'nodejs'
```

(from https://github.com/nodejs/security-wg/blob/f95d494df2101c36b25cdcbb62b75f6cc59a3251/.github/workflows/ossf-scorecard-reporting.yml#L24-L38)

The action will iterate through the database file and make a call to the public Scorecard Webapp to get the latest score of the projects in the database file. The action then updates the scores in the database file, creates an updated report and pushes it back to its original location in the users GitHub repository. The action can also optionally create issues in the users repository about any negative changes to scores in any of the projects.



Interface enumeration

Scorecard Monitor has the following interfaces:

#	Interface	Inbound data	Outbound data
1	GitHub Repository that contains database, scope and report files	Scope, DB & report files	Scope, DB & report files
2	Scorecard webapp	Response status none	POST request with scope, DB & report files.

Sensitive data

Scorecard Monitor processes the following sensitive data:

#	Name	Level of sensitivity	Documentation
1	GitHub Token	Moderate	https://docs.github.com/en/actions/security-for-github-actions/security-guides/automatic-token-authentication
2	Scorecard results	Moderate	

The table only includes data that the application processes. It does not include data about its environment, the user, files on the system etc., which can be sensitive.

Attack impact

Examples of ways malicious threat actors can compromise Scorecard Monitor security.

#	Description	Impact
1	Deny Scorecard Monitor of service	Prevents Scorecard Monitor from uploading scorecard results which in turn could impact business logic or allow an attacker to hide issues in packages to compromise them.
2	Command execution	1) Same as above. 2) Overtake CI and potentially use its resources. 3) Upload wrong results. 4) Leak sensitive data.

Scorecard

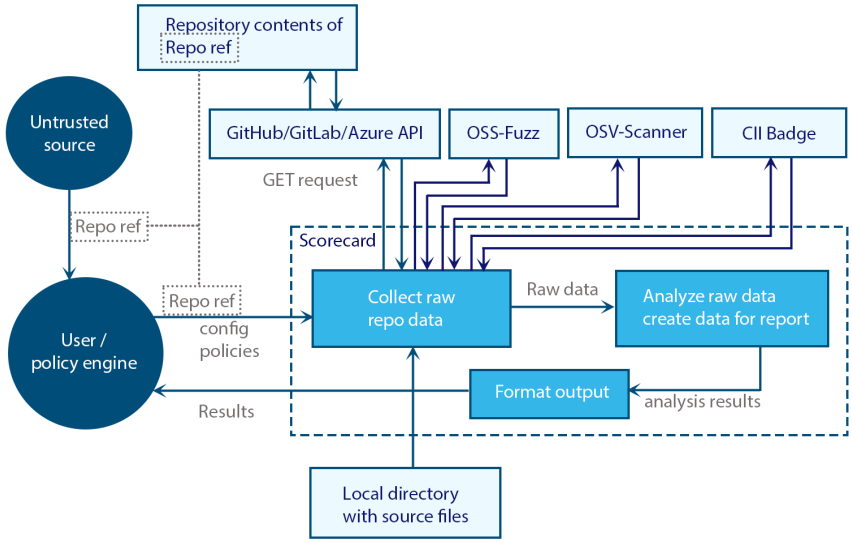
URL: https://github.com/ossf/scorecard

Scorecard is a command line tool (and library) that allows users to analyse code repositories for supply-chain risk. Users invoke Scorecard with a repository reference and configurations and policies. The repository reference can be untrusted and the configuration and policies are always trusted. The repository reference can be to either a remotely hosted repository at GitHub, Gitlab or Azure Devops, or it can be a path to a local directory.

Scorecard first pulls in the raw data from the repository. If Scorecard is used with a reference to a remote code repository, Scorecard will call the API of the repository host which returns data about the repository.

Like the repository ref, the contents of the repository can also be untrusted.

After pulling in the raw data from the code repository, Scorecard analyzes the raw data, formats the findings to an output of the user's choice and returns the formatted output.



Interface enumeration

Scorecard has the following interfaces:

#	Interface	Inbound data	Outbound data
1	User / policy engine	1) Repo ref/dir path. 2) Config 3) Policies	Scorecard results
2	GitHub/Gitlab/Azure API	Repository data and contents	Repo ref
3	Local file system	Source code	none

Sensitive data

Scorecard processes the following sensitive data:

#	Name	Level of sensitivity	Documentation
1	Users GitHub Token	Moderate	https://github.com/ossf/scorecard?tab=readme-ov-file#authentication
2	Scorecard results	Moderate	

The table only includes data that the application processes. It does not include data about its environment, the user, files on the system etc., which can be sensitive.

Attack impact

Examples of ways malicious threat actors can compromise Scorecards security.

#	Description	Impact
1	Deny Scorecard of service	Prevents Scorecard from performing analysis which in turn could impact business logic or allow an attacker to hide issues in packages to compromise them.
2	Command execution	1) Same as above. 2) Upload wrong results. 3) Leak sensitive data. 4) Install malware
3	MitM attack between Scorecard and user	Present wrong results to the user.

Use cases

Here we describe some common use cases.

1. Scorecard as a command-line tool: This is a popular use case.
2. Scorecard is used to filter out risky libraries: Some users adopt Scorecard as a part in their defence to reject risky libraries in the dependency tree. This can be as part of a middleware or admission controller that checks dependencies and either reject or approve the request based on Scorecards results. Scorecard can here run as a continuous process or it can run in an ephemeral environment.
3. Continuous monitoring: Users can adopt Scorecard to continuously monitor their own and 3rd-party libraries. As such, they run Scorecard on their own infrastructure and store the results on their own infrastructure.

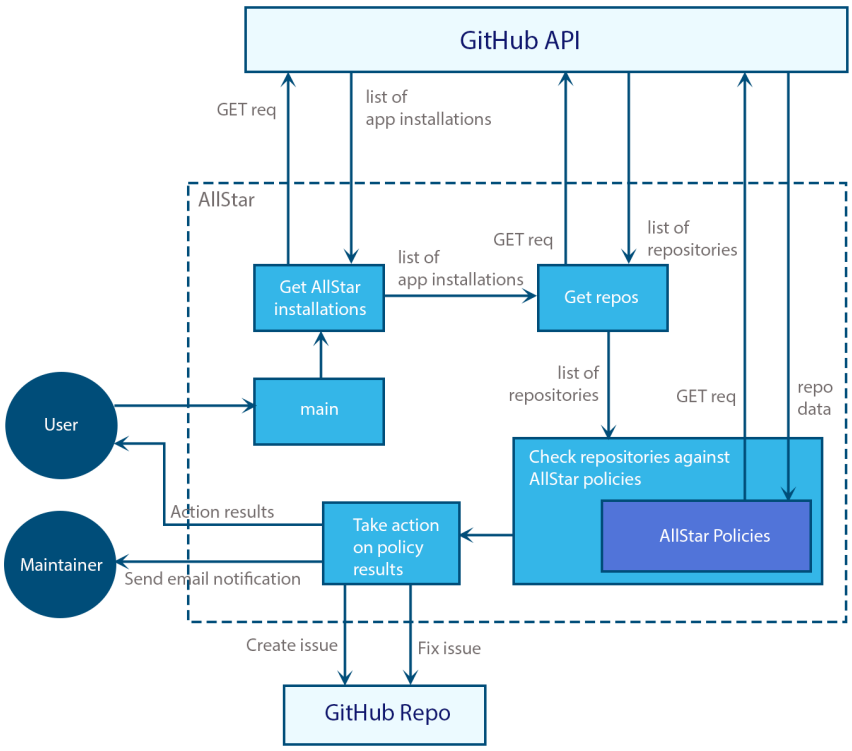
In all use cases, Scorecard processes untrusted data from the source tree of the project under analysis. Bugs in Scorecards processing of this data are the most critical potntial point of failure in Scorecard. For an attacker to be able to exploit such bugs, they need to have a deterministic way to trigger a privilege escalation from their repository. Furthermore, once an attacker knows of vulnerabilities they can exploit in such a manner, they can in most common use cases not trigger the attack when they want; they will have to configure their repository in a way that would exploit a vulnerability and then wait for the user or an automated process to invoke Scorecard which would then launch the attack. An example of such an attack could be that the owner of a repository is able to prevent the Scorecard process from working as intended. For example, a user may be monitoring all the projects their organizations SBOMs. By way of an example, let us consider that the list has 1,000 projects. An attacker may configure project number 100 to stop Scorecard from processing the remaining projects on the list. Meanwhile the attacker configures several projects in such a way that they can harm users. Since the Scorecard process is blocked, it will not alert such configuration. Another example is an attack where the contents of the project under analysis is able to compromise the contents of the filesystem outside of the paths that Scorecard uses; Scorecard downloads the repository contents by way of the platform APIs that return a compressed file that Scorecard that decompresses into a directory that Scorecard has assigned for this purpose. The repository contents should not be able to exceed this directory.

Allstar

URL: https://github.com/ossf/allstar

Allstar is a GitHub app that monitors and analyzes GitHub repositories for best practices of security supply-chain. Allstar implements a series of policies that analyze its repositories. Users can choose to either log the analysis results, create GitHub issues and/or email the findings to users. Allstar is installed at the organization level, repository level or policy level.

1. First, a user invokes the main entrypoint [ref]
2. Allstar then fetches all the Allstar installations in scope. It does so with a GET request to the GitHub API [ref]
3. Allstar then retrieves a list of repositories from the list of Allstar installations. It does so with a call to the GitHub API. [ref]
4. Allstar then runs its policies on all the repositories. The policies fetch data about the repositories from the GitHub API ad hoc. [ref]
5. Allstar outputs a series of actions based on the results of the policy analysis. If a check fails, the user has configured an action to take which can be to output the results to the user, create an issue in a GitHub repository or send an email to an administrator [ref].
6. Allstar then takes the actions and returns whether those actions were successful.



Interface enumeration

Allstar has the following interfaces:

#	Interface	Inbound data	Outbound data
1	User	Configuration	Action results
2	GitHub API	Repository data and contents	GET req
3	GitHub repository	Action result	
4	Maintainer	None	Action results

Sensitive data

Allstar processes the following sensitive data:

#	Name	Level of sensitivity	Documentation
1	Policy analysis results	Moderate	
2	Data about which policies are enabled	Low	
3	GitHub token	Moderate	Stored in the `SECRET_TOKEN` env
4	Private key	High	Stored in the `PRIVATE_KEY_PATH` env

#	Name	Level of sensitivity	Documentation
5	GitHub installation token	Moderate	https://github.com/ossf/allstar/blob/0235c413ddc6b3a0d91a2970f62df295cc6929e5/pkg/scorecard/scorecard.go#L150-L155
6	Email addresses	Low-moderate	These are PII and could allow malicious threat actors to enumerate users with high privileges.
7	Repositories in scope	Low-moderate	Lower-privileged users should not be able to enumerate repositories in scope.
8	Repository data	Moderate	Allstar fetches data from repositories in scope such as branch names and branch protection settings.

The table only includes data that the application processes. It does not include data about its environment, the user, files on the system etc., which can be sensitive.

Attack impact

Examples of ways malicious threat actors can compromise Allstars security.

#	Description	Impact
1	Deny Scorecard of service	Prevents Allstar from performing analysis which in turn could impact business logic or allow an attacker to hide issues in packages to compromise them.
2	Command execution	1) Same as above. 2) Upload wrong results. 3) Leak sensitive data. 4) Install malware. 5) Change policies. 6) Hijack compute resources.
3	MitM attack between Allstar and user	Present wrong results to the user.

Fuzzing

During the audit, we added a fuzz test for the Scorecard probes. It covers most probes except for a few select ones that are just too simple to dedicate efforts on; these are probes that The fuzz test invokes a single probe in each iteration; it chooses that probe to test, then generates the raw data that the given probe processes and then invokes the probe. We added the fuzzer to Scorecard OSS-Fuzz integration. At the time of completion of this audit, OSS-Fuzz has run the fuzzer for 1,572 CPU hours. A problem with fuzzing Scorecard probes is that each probe relies on a yaml file that contains metadata for the probe. For example, the codeReviewOneReviewers probe defines the following `def.yml` file: https://github.com/ossf/scorecard/blob/main/probes/codeReviewOneReviewers/def.yml. In the context of fuzzing, this is a problem because fuzzers do not have access to the same file system that they have access to during build time. Specifically, when OSS-Fuzz builds the Scorecard fuzzers, we have the Scorecard repository in the container, whereas when OSS-Fuzz runs the fuzzer, we should expect an empty filesystem. We had to make some modifications to the probes. Essentially, at build time, we add a global variable in each probe containing the YAML from the `def.yml` file. Next, at runtime, the fuzzers `init` function writes the contents of the YAML from each probe to the file system. The fuzzer and the OSS-Fuzz build script reflects this behavior.

With the completion of this audit, the fuzzer still runs continuously by way of OSS-Fuzz.

The following OSS-Fuzz commits were part of this audit:

1. https://github.com/google/oss-fuzz/commit/59ccdf4438d3a148c149bb4df8168ebac2787733
2. https://github.com/google/oss-fuzz/commit/14bf34015fcfdaa62e9a01cb99ef60c502817335
3. https://github.com/google/oss-fuzz/commit/86bba1c44ab799c5b3fbdd4c46b959f4df1362fa
4. https://github.com/google/oss-fuzz/commit/c8cd4a3e2410136a0da7142a41caad6b6e504c66
5. https://github.com/google/oss-fuzz/commit/125419644d02f75b124ad9a014c5f6c25d5df2ad

Found issues

This section contains the issues that have surfaced from the manual auditing and fuzz testing of Scorecard. Some of the issues are more mature in their analysis than others depending on the impact analysis they have undergone.

ID	Name	Severity	Status
ADA-SCORC-2025-1	Allstars reviewbot uses insecure secret token by default	Moderate	Fixed
ADA-SCORC-2025-2	Scorecard infrastructure lacks hardened security context	Moderate	Reported
ADA-SCORC-2025-3	Anti-path traversal bypass when using Scorecard with localdir client	Low	Fixed
ADA-SCORC-2025-4	Large repo files can exhaust memory	Low	Reported
ADA-SCORC-2025-5	Services run with root privileges in containers	Low	Reported
ADA-SCORC-2025-6	Repositories did not require reviews for PRs to get merged	Low	Fixed
ADA-SCORC-2025-7	Cron Webhook reads post requests body entirely into memory	Low	Fixed
ADA-SCORC-2025-8	Dependency to Scorecard attestor uses deprecated crypto library	Informational	Reported
ADA-SCORC-2025-9	Nil-pointer dereference in issueActivityByProjectMember probe	Informational	Fixed

Allstars reviewbot uses insecure secret token by default

Severity	Moderate
Status	Fixed
id	ADA-SCORC-2025-1
Threat	CWE-798: Use of Hard-coded Credentials

Allstars reviewbot uses a plaintext secret token that requires change in the source code to alter:

https://github.com/ossf/allstar/blob/294ae985cc2facd0918e8d820e4196021aa0b914/pkg/reviewbot/reviewbot.go#L57-L67

```
57  func (h *WebookHandler) HandleRoot(w http.ResponseWriter, r *http.Request) {
58      // Validate payload
59      payload, err := github.ValidatePayload(r, []byte(secretToken))
60      if err != nil {
61          log.Error().Interface("payload", payload).Err(err).Msg("Got an invalid payload"
                )
62          w.WriteHeader(400)
63          if _, err := fmt.Fprintln(w, "Got an invalid payload"); err != nil {
64              log.Error().Err(err).Msg("Failed to write http response")
65          }
66          return
67      }
```

The `secretToken` variable is defined globally in the source code:

https://github.com/ossf/allstar/blob/294ae985cc2facd0918e8d820e4196021aa0b914/pkg/reviewbot/reviewbot.go#L11

```
11  const secretToken = "FooBar"
```

By definition of CWE 798, this is a vulnerability:

"Inbound: the product contains an authentication mechanism that checks the input credentials against a hard-coded set of credentials. In this variant, a default administration account is created, and a simple password is hard-coded into the product and associated with that account. This hard-coded password is the same for each installation of the product, and it usually cannot be changed or disabled by system administrators without manually modifying the program, or otherwise patching the product. It can also be difficult for the administrator to detect."

https://cwe.mitre.org/data/definitions/798.html

GitHub security advisory has been published for this issue: https://github.com/ossf/allstar/security/advisories/GHSA-33f4-mjch-7fpr.

The issue has been fixed with e004ecb540d63ca6f5b1689b41af6c0040a82c73

Scorecard infrastructure lacks hardened security context

Severity	Moderate
Status	Reported
id	ADA-SCORC-2025-2
Threat	CWE-250: Execution with Unnecessary Privileges, CWE-276: Incorrect Default Permissions

A security context in Kubernetes defines the security-related attributes applied to a Pod or Container, controlling how it interacts with the system and what level of access it has during execution.

By default, Kubernetes does not apply a hardened security context to workloads, and users should configure their workloads to avoid an over permissive security context.

At a minimum, the following settings in the security context should be hardened.

runAsNonRoot

`runAsNonRoot` ensures that resources run without root privileges. Scorecard should approach this by setting it to true for all resources and by default and only remove as needed.

There are several risks associated with allowing a resource to run as root in the cluster. An attacker who compromises the root-privileged application has a wider attack surface to further escalate their privileges than they would if the resource did not have root privileges. In addition, some container breakout vulnerabilities will allow an attacker to obtain root privileges on the host if successfully exploited.

If an attacker compromises a Kubernetes container configured with `runAsNonRoot=false`, they gain root access inside the container, which dramatically increases the potential impact of the breach. With root privileges, the attacker can:

1. Escalate privileges by exploiting kernel vulnerabilities or using setuid binaries to break out of the container and gain control over the host node.
2. Access and modify sensitive files, both inside the container and on mounted host volumes, potentially tampering with configurations, credentials, or logs.
3. Interact with the container runtime or Docker socket (if mounted), enabling them to control other containers, deploy malicious workloads, or escape into the cluster.
4. Access service account tokens and secrets, allowing lateral movement to other pods or direct interaction with the Kubernetes API for further compromise.
5. Bypass security controls like file permissions or audit logging, making detection and containment more difficult.

In short, a compromised root container can serve as a launchpad for full host takeover, data exfiltration, and broader cluster-level attacks, especially in poorly secured environments.

Seccomp

Seccomp restricts the system calls (syscalls) a container can make to the host kernel. It's a layer of defense to limit what code inside a container can do, especially in case the container gets compromised.

Not setting a seccomp profile for Kubernetes containers exposes workloads to several potential security vulnerabilities, particularly around syscall abuse and kernel-level attack surfaces. As such, a missing seccomp profile is a security misconfiguration that increases the kernel attack surface and the risk from container escapes.

Kubernetes best practices for Pod hardening dictate that the Seccomp profile is either RuntimeDefault (recommended) or Localhost.

If an attacker compromises a Kubernetes container that is running without a seccomp profile, they can execute any available Linux system call, significantly increasing the risk of exploitation. This unrestricted syscall access allows the attacker to:

Exploit kernel vulnerabilities using dangerous syscalls like unshare, ptrace, or clone, potentially leading to container escape and host compromise. Bypass security boundaries that would otherwise block risky operations (e.g., creating new namespaces or manipulating kernel memory). Run advanced malware or reverse shells that rely on specific syscalls often restricted by seccomp, making it easier to establish persistence or exfiltrate data. Avoid detection, since seccomp profiles also provide syscall filtering and logging, which help in identifying malicious behavior.

Overall, not using a seccomp profile removes a critical layer of sandboxing, making it far easier for a compromised container to escalate privileges, attack the host, or pivot deeper into the Kubernetes cluster.

SELinux

Not configuring SELinux for Kubernetes containers significantly weakens the security posture of the cluster by eliminating a critical layer of mandatory access control (MAC). Without SELinux, containers rely solely on discretionary access control mechanisms like traditional Linux file permissions, which are insufficient in protecting against privilege escalation, container breakout, or unauthorized access to sensitive resources. SELinux enforces strict policies that limit what processes can do—even if they are running as root—helping to contain compromised containers and prevent them from affecting the host or other containers. Without it, any container that gains elevated privileges or misuses mounted host paths (such as `/var/run/docker.sock` or `/etc`) can potentially tamper with the host system or escalate to full control of the node. Moreover, the absence of SELinux removes fine-grained control over inter-process communication and file access, increasing the risk of lateral movement within the cluster. This lack of isolation also hinders compliance with security standards and benchmarks, making the environment more vulnerable to sophisticated attacks.

Linux capabilities

In Kubernetes, capabilities in the securityContext are a way to fine-tune the privileges granted to a container process, offering more control than simply running as root or non-root. They are part of Linux's capability-based security model, which breaks down the all-powerful root privileges into smaller, more manageable pieces.

Linux capabilities in Kubernetes allows enforcing the principle of least privilege at a granular level, which significantly reduces the risk of containerized workloads performing unauthorized or dangerous actions. By default, containers inherit a broad set of capabilities that may not be necessary for their intended function, which exposes the system to potential abuse if the container is compromised. For example, a container with capabilities like `CAP_NET_ADMIN` or `CAP_SYS_ADMIN` could manipulate network settings or interact with kernel-level features and thereby increase the likelihood of privilege escalation or host compromise. Carefully controlling which capabilities are added or dropped limits what a process can do, even if it needs root creates a tighter security boundary and makes it much harder for attackers to exploit misconfigurations or vulnerabilities within the container or the host system.

The best practice for managing capabilities in Kubernetes is to adopt a "deny by default" approach by explicitly dropping all capabilities and only adding back those that are strictly necessary for the container's functionality. This is typically done by setting `drop: ["ALL"]` in the container's securityContext, which removes all Linux capabilities—even those normally granted to the root user—ensuring that the container starts with the minimal privilege set. If the application requires specific capabilities to operate, such as `NET_BIND_SERVICE` to bind to a low-numbered port, these can be selectively re-added using the add field. This approach reduces the container's attack surface by preventing unnecessary privileges that could be exploited if the container is compromised. Implementing minimal capabilities, along with other hardening techniques like `runAsNonRoot`, seccomp profiles, and SELinux or AppArmor, forms a strong, layered defense strategy for securing Kubernetes workloads.

Privilege escalation

The `allowPrivilegeEscalation` setting in the Kubernetes security context determines whether a process running in a container can gain more privileges than its parent process, such as through the use of `setuid` or `setgid` binaries. If this option is set to true or left unset (as it defaults to true), it opens the door for attackers or compromised applications to escalate their privileges within the container. This can lead to scenarios where a non-root process spawns a child process with elevated capabilities, potentially bypassing security controls and gaining access to sensitive resources or performing restricted operations. Such privilege escalation can be particularly dangerous when combined with other misconfigurations, such as overly permissive Linux capabilities or mounted host paths, enabling attackers to break out of the container and affect the host or other workloads. Disabling privilege escalation (`allowPrivilegeEscalation: false`) is a critical defense-in-depth measure that helps enforce the principle of least privilege, preventing unauthorized access and reducing the impact of any potential security breach.

We recommend disallowing privilege escalation by default and instead let users enable it if they so require in their particular deployments. By disallowing privilege escalation by default, Scorecard is hardened with best practices in its default installation.

The following resources lack a security context:

1. https://github.com/ossf/scorecard/blob/main/cron/k8s/worker.yaml
2. https://github.com/ossf/scorecard/blob/main/cron/k8s/worker.release.yaml
3. https://github.com/ossf/scorecard/blob/main/cron/k8s/webhook.release.yaml
4. https://github.com/ossf/scorecard/blob/main/cron/k8s/transfer.yaml
5. https://github.com/ossf/scorecard/blob/main/cron/k8s/transfer.release.yaml
6. https://github.com/ossf/scorecard/blob/main/cron/k8s/transfer.release-raw.yaml
7. https://github.com/ossf/scorecard/blob/main/cron/k8s/transfer-raw.yaml
8. https://github.com/ossf/scorecard/blob/main/cron/k8s/controller.yaml
9. https://github.com/ossf/scorecard/blob/main/cron/k8s/controller.release.yaml
10. https://github.com/ossf/scorecard/blob/main/cron/k8s/cii.yaml
11. https://github.com/ossf/scorecard/blob/main/cron/k8s/cii.yaml

K8s docs on security context standards:

1. https://kubernetes.io/docs/tasks/configure-pod-container/security-context/
2. https://kubernetes.io/docs/concepts/security/pod-security-standards/

Anti-path traversal bypass when using Scorecard with localdir client

Severity	Low
Status	Fixed
id	ADA-SCORC-2025-3
Threat	CWE-61: UNIX Symbolic Link (Symlink) Following, CWE-23: Relative Path Traversal

Scorecard allows the localdir client to traverse the file system. If the repository contains symlinked files, Scorecard will follow the symlink and read the file the symlink points to. This is only an issue when using the localdir client.

This is an issue in odd use cases where the repository is not cloned from GitHub, GitLab or Azure Devops. Scorecard ignores symlinks in those cases at the client level. However, if the user creates the source directory on the local file system without Scorecards repository clients. For example, if the user clones the repository themselves and then runs Scorecard using `--local`, Scorecard will follow symlinks in a manner that could lead to an out of bounds read escalation which could leak sensitive data.

https://github.com/ossf/scorecard/blob/e29967dc475c1414ba740e3413ab543951c22754/clients/localdir/client.go#L167-L175

```
167  func getFile(clientpath, filename string) (*os.File, error) {
168    // Note: the filenames do not contain the original path - see ListFiles().
169    fn := path.Join(clientpath, filename)
170    f, err := os.Open(fn)
171    if err != nil {
172      return nil, fmt.Errorf("open file: %w", err)
173    }
174    return f, nil
175  }
```

This is a minor privilege escalation in that the repository owner, the user invoking Scorecard and the service responsible for running Scorecard can have different privilege levels. For example, a service may only allow users to view files the repository they have cloned, whereas Scorecard has permissions to read files in all cloned repositories on the system - including those that other users have cloned. As such, UserA can clone a repository to `/repositories/UserA/repo`. The repository contains a symlink to a file in `/repositories/UserB/repo`, and because Scorecard has permissions to read files in both directories, UserA may be able to leak sensitive data about UserB's cloned repository.

We recommend not following symlinks when reading files in the source trees under analysis.

The issue has been fixed with aa9d2230755d055f1a29c56a907eddcf82b3c4c7

Large repo files can exhaust memory

Severity	Low
Status	Reported
id	ADA-SCORC-2025-4
Threat	CWE-770: Allocation of Resources Without Limits or Throttling

https://github.com/ossf/scorecard/blob/main/checks/raw/shell_download_validate.go implements a suite of utility functions for parsing shell-scripts and reason about un-pinned references such e.g. un-pinned package-manager installs. Scorecard uses these utilities on the source files of the repository it is analysing and later it runs checks and probes against the results of the parsing. As such, the source files being parsed by the utilities in https://github.com/ossf/scorecard/blob/main/checks/raw/shell_download_validate.go are untrusted.

We found several cases in the utility functions of `strings.Split()` that could allow shell scripts formatted specifically to hurt Scorecard to exhaust memory. Essentially, `strings.Split()` can consume large ammounts of memory if the first parameter contains a high ammount of the pattern in the second parameter. For example, `strings.Split()` can exhaust memory on the following line:

https://github.com/ossf/scorecard/blob/ab2f6e92482462fe66246d9e32f642855a691dc1/checks/raw/shell_download_validate.go#L506

```
506      parts := strings.Split(pkg, "@")
```

… when it meets a line like this:

```
1  go get github.com/a/b(@*3000000000)latest ("@" repeated 3 billion times)
```

This is not an issue when analysing repositories with the `--repo` flag under normal circumstances. There may be exceptions for private Git servers, enterprise users or possibly git large file storage. The reason is that GitHub, Gitlab and probably Azure Devops set a limit of 100 MiB per file which is not enough to make Scorecard exhaust memory. As such, Scorecard is under normal circumstances protected by the hosting platforms' size limits, and this is an issue when running Scorecard with `--local` and perhaps some edge use cases.

For example, from a use case perspective: If an attacker can cause harm only in cases where the user is in charge of cloning the repository and then uses `--local` , is that a security issue or should the user verify such things before themselves?

We have found the following places to have potential to harm Scorecard in a similar manner:

https://github.com/ossf/scorecard/blob/ab2f6e92482462fe66246d9e32f642855a691dc1/checks/raw/shell_download_validate.go#L709-L712

```
709  func isChocoUnpinnedDownload(cmd []string) bool {
710    // If this is an install command, then some variant of requirechecksum must be
         present.
711    for i := 2; i < len(cmd); i++ {
712      parts := strings.Split(cmd[i], "=")
```

https://github.com/ossf/scorecard/blob/ab2f6e92482462fe66246d9e32f642855a691dc1/checks/raw/shell_download_validate.go#L1326-L1328

```
1326    line = line[2:]
1327    for _, name := range shellsToMatch {
1328      parts := strings.Split(line, " ")
```

https://github.com/ossf/scorecard/blob/ab2f6e92482462fe66246d9e32f642855a691dc1/checks/raw/shell_download_validate.go#L314-L317

```
314    for _, u := range urls {
315      // Look for a URL of the form: https://raw.githubusercontent.com/{owner}/{repo}/{
           ref}/{path}
316      if u.Scheme == "https" && u.Host == "raw.githubusercontent.com" {
317        segments := strings.Split(u.Path, "/")
```

https://github.com/ossf/scorecard/blob/ab2f6e92482462fe66246d9e32f642855a691dc1/checks/raw/permissions.go#L395-L402

```
395    for _, job := range workflow.Jobs {
396      for _, step := range job.Steps {
397        uses := fileparser.GetUses(step)
398        if uses == nil {
399          continue
400        }
401        // remove any version pinning for the comparison
402        uses.Value = strings.Split(uses.Value, "@")[0]
```

We recommend checking the length of the first parameter before calling with with `strings.Split()`.

Services run with root privileges in containers

Severity	Low
Status	Reported
id	ADA-SCORC-2025-5
Threat	CWE-272: Least Privilege Violation, CWE-250: Execution with Unnecessary Privileges

The following dockerfiles do not specify a user which defaults to the root user

From the Docker documentation:

“Remember, if you don’t set a USER in your Dockerfile, the user will default to root. Always explicitly set a user, even if it’s just to make it clear who the container will run as.”

https://www.docker.com/blog/understanding-the-docker-user-instruction.

This may be overprivileged; since none of the Dockerfiles specify a user, we assume that they have not undergone analysis of whether root user is required.

The following Dockerfiles do not define a user and default to the root user. Unless this is necessary for the application, the Dockerfiles should add a lower-privileged user.

1. https://github.com/ossf/scorecard/blob/main/cron/internal/webhook/Dockerfile
2. https://github.com/ossf/scorecard/blob/main/cron/internal/worker/Dockerfile
3. https://github.com/ossf/scorecard/blob/main/cron/internal/controller/Dockerfile
4. https://github.com/ossf/scorecard/blob/main/cron/internal/cii/Dockerfile
5. https://github.com/ossf/scorecard/blob/main/cron/internal/bq/Dockerfile
6. https://github.com/ossf/scorecard/blob/main/clients/githubrepo/roundtripper/tokens/server/Dockerfile
7. https://github.com/ossf/scorecard/blob/main/attestor/Dockerfile
8. https://github.com/ossf/scorecard/blob/main/Dockerfile
9. https://github.com/ossf/scorecard-action/blob/main/Dockerfile
10. https://github.com/ossf/scorecard-webapp/blob/main/Dockerfile

If any of these applications need root privileges, we still recommend assigning a user to avoid confusion over required permissions in the future.

Repositories did not require reviews for PRs to get merged

Severity	Low
Status	Fixed
id	ADA-SCORC-2025-6
Threat	SLSA v0.1 threat A1

We found 1 case of pull requests being merged without receiving maintainer approval where a pull request received no approvals prior to being merged. The particular cases is not problematic, but it demonstrate that the repository's branch protection settings are not configured in a way that pull requests require maintainer approval to be merged.

Refs: 1. https://github.com/ossf/scorecard-monitor/pull/69

The Scorecard team has reviewed the repository settings and have ensured that pull requests must be reviewed before getting merged.

Cron Webhook reads requests body entirely into memory

Severity	Low
Status	Fixed
id	ADA-SCORC-2025-7
Threat	CWE-770: Allocation of Resources Without Limits or Throttling

The cron webhooks main purpose is to tag the following five images:

1. gcr.io/openssf/scorecard-batch-controller
2. gcr.io/openssf/scorecard-batch-worker
3. gcr.io/openssf/scorecard-cii-worker
4. gcr.io/openssf/scorecard-bq-transfer
5. gcr.io/openssf/scorecard-github-server

source: https://github.com/ossf/scorecard/blob/ab2f6e92482462fe66246d9e32f642855a691dc1/cron/internal/webhook/main.go#L34-L40

The cron webhook starts an HTTP server on port 8080 with a single route. When that handler receives a `POST` request, it reads the JSON payload from the request body and unmarshals it into a `ShardMetadata` protobuf message. Next, it constructs a Google Cloud authenticator using a . It then and then loops over a fixed set of images and applies the `stable` tag to the image identified by the commit SHA from the incoming metadata. After tagging all images successfully, it returns a “200 OK” response.

When the webhook handler receives a request, it reads it entirely into memory in an unbounded manner. This can be exploited by an attacker who can send HTTP requests to the handler; they can send an HTTP request containing a large body which will exhaust the servers memory and make the webhook unavailable to other users.

https://github.com/ossf/scorecard/blob/e51d93e9022b61976836d584f4e1de0564235860/cron/internal/webhook/main.go#L43-L46

```
43  func scriptHandler(w http.ResponseWriter, r *http.Request) {
44      switch r.Method {
45      case http.MethodPost:
46          jsonBytes, err := io.ReadAll(r.Body)
```

We recommend limiting the ammount of bytes the webhook handler will read from the incoming request. This aligns with the webhooks purpose: It does not need to receive and process large requests.

The issue has been fixed with c29a04d46d1570393e94662bc34e9906398e1bfa

Dependency to Scorecard attestor uses deprecated crypto library

Severity	Informational
Status	Reported
id	ADA-SCORC-2025-8
Threat	CWE-477: Use of Obsolete Function

The Scorecard attestor uses the attestlib library to sign images which uses the deprecated golang.org/x/crypto/openpgp library. golang.org/x/crypto/openpgp is deprecated and unmaintained. The documentation reports that security fixes are still received and considered for the library, however, from internal discussions that the Go team strongly encourages users to discontinue using this library. Over time this could have both reliability and security implications for Scorecard.

Scorecard uses the 3rd party dependency in the following place:

https://github.com/ossf/scorecard/blob/417363bc2111719e480d8152e1cfb824fb744ba1/attestor/command/sign.go#L67-L77

```
67      case pgpPriKeyPath != "":
68          logger.Info("Using pgp key for signing.")
69          signerKey, err := os.ReadFile(pgpPriKeyPath)
70          if err != nil {
71              return fmt.Errorf("fail to read signer key: %w", err)
72          }
73          // Create a cryptolib signer
74          cSigner, err = attestlib.NewPgpSigner(signerKey, pgpPassphrase)
75          if err != nil {
76              return fmt.Errorf("creating pgp signer failed: %w", err)
77          }
```

This dependency uses the deprecated x crypto library to create the signer:

https://github.com/grafeas/kritis/blob/3348562cec04c175bd0fdf7cbd4e26ed73797e71/pkg/attestlib/pgp.go#L19-L27

```
19  package attestlib
20
21  import (
22      "bytes"
23      "fmt"
24      "io/ioutil"
25
26      "github.com/pkg/errors"
27      "golang.org/x/crypto/openpgp"
28      "golang.org/x/crypto/openpgp/armor"
29  )
```

https://github.com/grafeas/kritis/blob/3348562cec04c175bd0fdf7cbd4e26ed73797e71/pkg/attestlib/pgp.go#L78-L82

```
78  func NewPgpSigner(privateKey []byte, passphrase string) (Signer, error) {
79      keyring, err := openpgp.ReadArmoredKeyRing(bytes.NewReader(privateKey))
80      if err != nil {
81          return nil, errors.Wrap(err, "error reading armored private key")
82      }
```

Further reading: 1. https://pkg.go.dev/golang.org/x/crypto/openpgp 2. https://github.com/golang/go/issues/44226

https://github.com/ProtonMail/go-crypto is a community-maintained fork of `x/crypto/openpgp`

Nil-pointer dereference in issueActivityByProjectMember probe

Severity	Informational
Status	Fixed
id	ADA-SCORC-2025-9
Threat	CWE-476: NULL Pointer Dereference

The probes fuzzer we added during the audit found a case where the issueActivityByProjectMember probe would dereference a nil-pointer leading to a panic. The issueActivityByProjectMember probe analyses project activity by members of the project. As part of that analysis it iterates through issues and issue comments in the repository and checks whether the author of either are associated with the project. Technically, the author association can be nil during this processing, but the probe does not check for a nil pointer before dereferencing it.

The issue was fixed in https://github.com/ossf/scorecard/pull/4642.

OSS-Fuzz issue: https://issues.oss-fuzz.com/issues/420740700

The issue has been fixed with 1d8f219d2dc83cf82f9938b0b06a774303d82421